



早稲田大学
WASEDA University

WASEDA University

Waseda Research Institute for Science and Engineering

Fundamentals of Programming

孫 鶴鳴

SUN Heming

May 8th



Outline

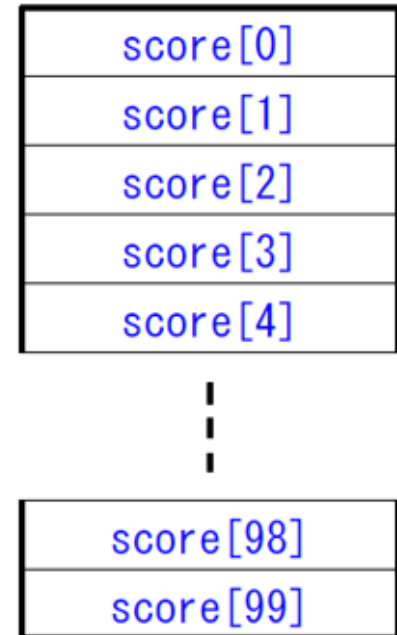
- ◆ Review of Array
- ◆ String

Array

- ```
int score[100];
```

int      score[100];  
↓            ↓  
type        number of size
- `score[0]...score[99]`: a block of 100 consecutive elements can be used. (`score[100]` is not available)
- Each element can be used as an int- variable
- The number within `[]` is called subscript
- The subscript can be used as a variable

Size: 100



```
int score[100];

score[10] = 75;
```

=

```
int score[100];
int count;

count = 10;
score[count] = 75;
```

# Examples of Array

```
int st[4]={10, 20, 30, 40};
```

```
int st2[]={10, 20, 30};
```

```
char a[4]={ 'A' , 'B' , 'C' , '¥0' };
```

```
Int p[3]={10, 20};
```

It mean  $p[0]=10$ ,  $p[1]=20$ ,  $p[2]=0$

# ASCII Table

## ◆ American Standard Code for Information Interchange

❖ 0-31: control characters

❖ 32-126: printable characters

❖ 127: delete code

ASCII Hex Symbol

|    |   |     |
|----|---|-----|
| 0  | 0 | NUL |
| 1  | 1 | SOH |
| 2  | 2 | STX |
| 3  | 3 | ETX |
| 4  | 4 | EOT |
| 5  | 5 | ENQ |
| 6  | 6 | ACK |
| 7  | 7 | BEL |
| 8  | 8 | BS  |
| 9  | 9 | TAB |
| 10 | A | LF  |
| 11 | B | VT  |
| 12 | C | FF  |
| 13 | D | CR  |
| 14 | E | SO  |
| 15 | F | SI  |

ASCII Hex Symbol

|    |    |     |
|----|----|-----|
| 16 | 10 | DLE |
| 17 | 11 | DC1 |
| 18 | 12 | DC2 |
| 19 | 13 | DC3 |
| 20 | 14 | DC4 |
| 21 | 15 | NAK |
| 22 | 16 | SYN |
| 23 | 17 | ETB |
| 24 | 18 | CAN |
| 25 | 19 | EM  |
| 26 | 1A | SUB |
| 27 | 1B | ESC |
| 28 | 1C | FS  |
| 29 | 1D | GS  |
| 30 | 1E | RS  |
| 31 | 1F | US  |

ASCII Hex Symbol

|    |    |         |
|----|----|---------|
| 32 | 20 | (space) |
| 33 | 21 | !       |
| 34 | 22 | "       |
| 35 | 23 | #       |
| 36 | 24 | \$      |
| 37 | 25 | %       |
| 38 | 26 | &       |
| 39 | 27 | '       |
| 40 | 28 | (       |
| 41 | 29 | )       |
| 42 | 2A | *       |
| 43 | 2B | +       |
| 44 | 2C | ,       |
| 45 | 2D | -       |
| 46 | 2E | .       |
| 47 | 2F | /       |

ASCII Hex Symbol

|    |    |   |
|----|----|---|
| 48 | 30 | 0 |
| 49 | 31 | 1 |
| 50 | 32 | 2 |
| 51 | 33 | 3 |
| 52 | 34 | 4 |
| 53 | 35 | 5 |
| 54 | 36 | 6 |
| 55 | 37 | 7 |
| 56 | 38 | 8 |
| 57 | 39 | 9 |
| 58 | 3A | : |
| 59 | 3B | ; |
| 60 | 3C | < |
| 61 | 3D | = |
| 62 | 3E | > |
| 63 | 3F | ? |

ASCII Hex Symbol

|    |    |   |
|----|----|---|
| 64 | 40 | @ |
| 65 | 41 | A |
| 66 | 42 | B |
| 67 | 43 | C |
| 68 | 44 | D |
| 69 | 45 | E |
| 70 | 46 | F |
| 71 | 47 | G |
| 72 | 48 | H |
| 73 | 49 | I |
| 74 | 4A | J |
| 75 | 4B | K |
| 76 | 4C | L |
| 77 | 4D | M |
| 78 | 4E | N |
| 79 | 4F | O |

ASCII Hex Symbol

|    |    |   |
|----|----|---|
| 80 | 50 | P |
| 81 | 51 | Q |
| 82 | 52 | R |
| 83 | 53 | S |
| 84 | 54 | T |
| 85 | 55 | U |
| 86 | 56 | V |
| 87 | 57 | W |
| 88 | 58 | X |
| 89 | 59 | Y |
| 90 | 5A | Z |
| 91 | 5B | [ |
| 92 | 5C | \ |
| 93 | 5D | ] |
| 94 | 5E | ^ |
| 95 | 5F | _ |

ASCII Hex Symbol

|     |    |   |
|-----|----|---|
| 96  | 60 | ` |
| 97  | 61 | a |
| 98  | 62 | b |
| 99  | 63 | c |
| 100 | 64 | d |
| 101 | 65 | e |
| 102 | 66 | f |
| 103 | 67 | g |
| 104 | 68 | h |
| 105 | 69 | i |
| 106 | 6A | j |
| 107 | 6B | k |
| 108 | 6C | l |
| 109 | 6D | m |
| 110 | 6E | n |
| 111 | 6F | o |

ASCII Hex Symbol

|     |    |   |
|-----|----|---|
| 112 | 70 | p |
| 113 | 71 | q |
| 114 | 72 | r |
| 115 | 73 | s |
| 116 | 74 | t |
| 117 | 75 | u |
| 118 | 76 | v |
| 119 | 77 | w |
| 120 | 78 | x |
| 121 | 79 | y |
| 122 | 7A | z |
| 123 | 7B | { |
| 124 | 7C |   |
| 125 | 7D | } |
| 126 | 7E | ~ |
| 127 | 7F |   |

# Outline

---

- ◆ Review of Array
- ◆ String

# Input Characters

```
int main(){
 char a[5] = {'t', 'e', 's', 't', '¥0'};
 printf("%s¥n", a);
}
```

The end of char array

# Input String

```
int main(){
 char a[5] = "test";
 printf("%s\n", a);
}
```

C supports string literal syntax:

A "string literal" is a sequence of characters from the source character set enclosed in double quotation marks (" "). String literals are used to represent a sequence of characters which, taken together, form a null-terminated string.



# Exercise 1

```
Char a[]="abcabc";
```

Printf:

Include a on 0

Include a on 3

# Exercise 1 (cont.)

```
#include<stdio.h>

int main()
{
 char a[10] = {'a', 'l', 'e', 'x', 'a', 'n',
 'd', 'e', 'a', 'r'};

 int i;
 for (i = 0; i < 10; i++) {
 if (a[i] == 'a') {
 printf("included a on %d¥n", i);
 }
 }
 return 0;
}
```

# Exercise 2

```
#include<stdio.h>

int main()
{
 char a[10] = {'a', 'l', 'e', 'x', 'a', 'n',
 'd', 'e', 'a', 'r'};

 int i;
 for (i = 0; i < 10; i++) {
 if (a[i] == 'a') {
 printf("included a on %d¥n", i);
 }
 }
 return 0;
}
```

a method to define a loop as while.

```
for (initialization_expression; loop_condition; increment_expression) {
 statement1;
 statement2;
 ...
 statementn;
}
```



- initialization\_expression is executed before execution of the loop starts.
- The execution of the loop continues until the loop\_condition is false.
- The increment\_expression is executed at the end of each loop iteration.

Try to use '¥0' in the loop condition

# Exercise 2 (cont.)

```
#include<stdio.h> char2.c

int main()
{

 char a[10] = {'a', 'l', 'e', 'x', 'a', 'g',
'g', 'g'};

 int i;

 for (i = 0; a[i] != '\0'; i++) {
if (a[i] == 'a') {
 printf("included a on %d\n", i);
 }
 }

 return 0;
}
```

# Exercise 3

```
#include<stdio.h>
```

char3.c

```
int main()
{
```

```
 char a[10] = {'a', 'l', 'e', 'x', '¥0',
 'a', 'g', 'a', 'g'};
```

```
 int i;
 for (i = 0; a[i] != '¥0'; i++) {
 if (a[i] == 'a') {
 printf("included on %d¥n", i);
 }
 }
```

How many 'a'  
is output?

```
 printf("%s¥n", a);
```

What is output?

```
 return 0;
```

```
}
```

# Some Useful Functions

- `#include <string.h>`
- `strcpy(a, b)`
- `strlen(a)`
- `strcmp(a, b)`

# strcpy

stringcp.c

```
#include <stdio.h>
#include <string.h>

int main(void)
{
 char str1[] = "ABCDEF";
 char str2[] = "123";
 char *p = "abcd";

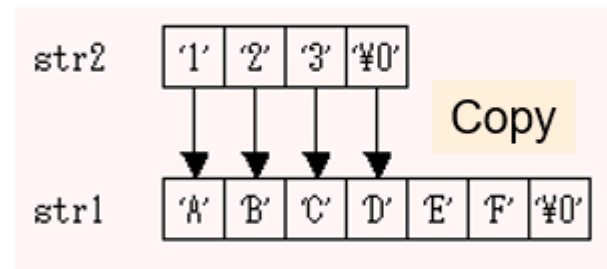
 strcpy(str1, str2); /* copy */
 printf("%s\n", str1);

 strcpy(str1, p);

 printf("%s\n", str1);

 strcpy(str1, "xyz"); printf("%s\n", str1);

 return 0;
}
```



# strlen

```
#include <stdio.h>
#include <string.h>
```

```
int main(void)
{
```

```
 int len;
 char s[] = "today";
```

```
 len = strlen(s);
```

Try to print out “today” is 5 character

```
 return 0;
```

```
}
```



# strlen (cont.)

```
#include <stdio.h>
#include <string.h>

int main(void)
{
 int len;
 char s[] = "today";

 len = strlen(s);

 printf("¥"%s¥" is %d character¥n", s, len);

 return 0;
}
```

# strcmp

```
int strcmp(const char *s1, const char *s2);
```

strcmp returns 0 when the strings are equal,  
a negative integer when  $s1 < s2$ ,  
a positive integer if  $s1 > s2$ ,

How to justify?

# Exercise 4

---

- ◆ Use strcpy to set the value for two strings
  - ✦ Str1: “abcdef”
  - ✦ Str2: “ABCDEF”
- ◆ Use strcmp to compare their values and output the result

# Exercise 4 (cont.)

```
#include <stdio.h>
#include <string.h>

int main ()
{
 char str1[15];
 char str2[15];
 int ret;

 strcpy(str1, "abcdef");
 strcpy(str2, "ABCDEF");

 ret = strcmp(str1, str2);

 if(ret < 0)
 {
 printf("str1 is less than str2\n");
 }
 else if(ret > 0)
 {
 printf("str2 is less than str1\n");
 }
 else
 {
 printf("str1 is equal to str2\n");
 }

 return(0);
}
```

# Exercise 5

---

- ◆ Please remove all occurrences of the character `c` from the string `s`
- ◆ Input string `s`: use `scanf`

for example

input: communication,

output: ommuniation